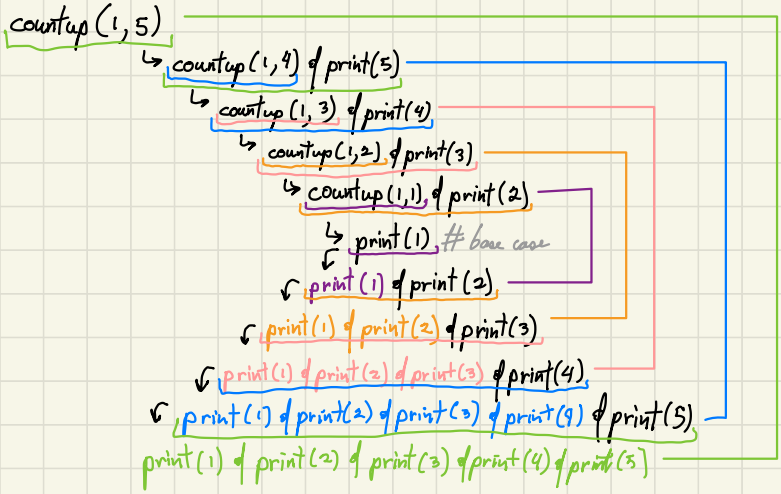
Recursion

Recursion

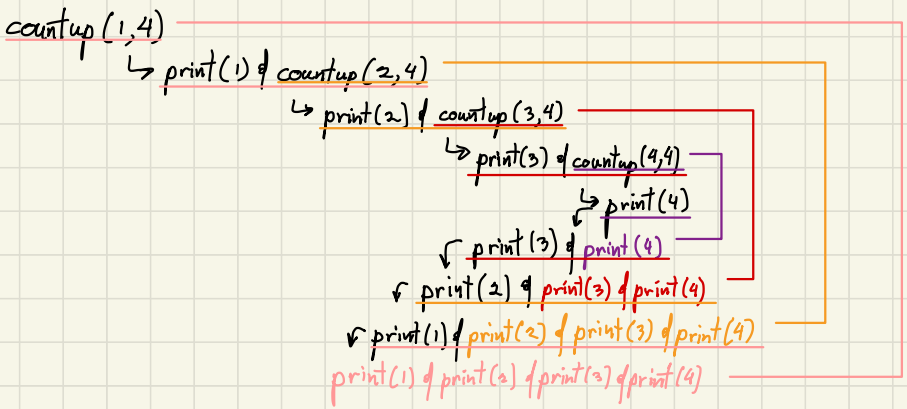
- Closely related to mathematical induction, where we define the sol'n as a combination of sol'ns to smaller instances of the same problem.

  - **Base Case**: The condition w/ the smallest possible input.

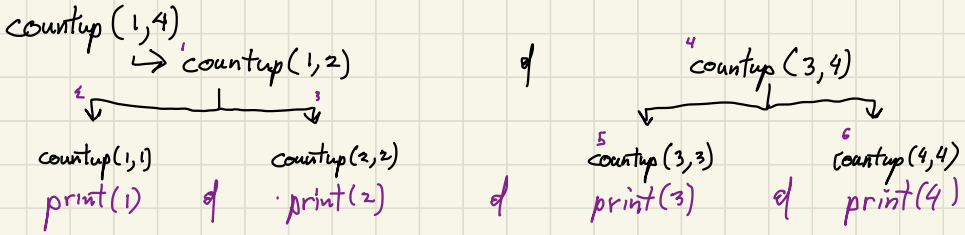
  - **Recursive Step**: An assumption that when calling the function w/ a smaller input, it'll do it's job. After that, find a way to use that assumption in order to solve the problem w/ the given input.



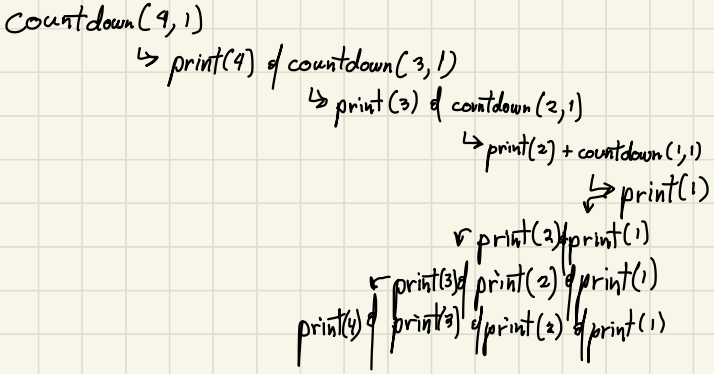

```python
def count_up(start, end):
    if start == end:
        # the base case
        print(start)
    else:
        # the recursive step
        count_up(start, end - 1)
        print(end)

count_up(1, 5)
```



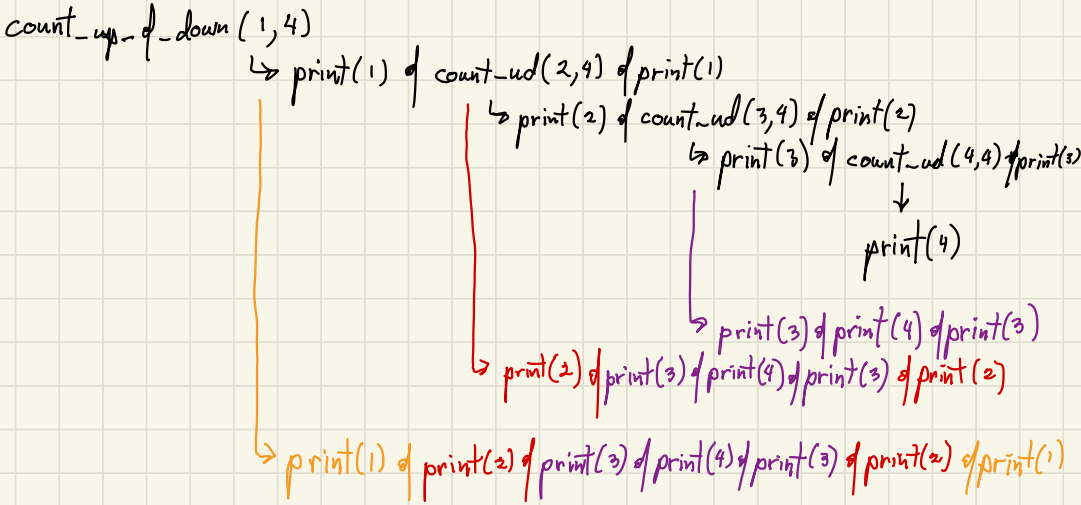

```
# assumes start <= end
# prints a sequence of
# ascending values, one per line

def count_up(start, end):
    if start == end:
        print(start)
    else:
        print(start)
        count_up(start + 1, end)

count_up(1, 4)
```




```
# assumes start <= end
# prints a sequence of ascending values, one per line

def count_up(start, end):
    if start == end:
        print(start)
    else:
        mid = (start + end) // 2
        count_up(start, mid)
        count_up(mid+1, end)

count_up(4)
```




```python
# assumes start <= end

def count_down(start, end):
    if start == end:
        print(start)
    else:
        count_down(start + 1, end)
        print(start)
```




```python
# assumes start <= end

def count_up_and_down(start, end):
        if start == end:
        print(start)
    else:
        print(start)
        count_up_and_down(start + 1, end)
        print(start)
```

Factorial

- Iterative:

$$n! = n * (n-1) * (n-2) * (n-3) * \dots * 3 * 2 * 1$$

- Recursive

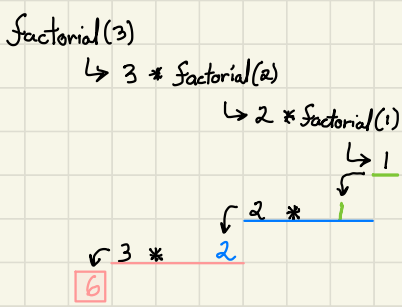
$$n! = n * \overbrace{(n-1) * (n-2) * (n-3) * \dots * 3 * 2 * 1}^{(n-1)!}$$

$$= n * (n-1)!$$

$$n! = \begin{cases} 1 & \text{if } n = 1 \\ n * (n-1)! & \text{if } n > 1 \end{cases}$$

- Base case: if $n = 1$, then return 1
- Hypothesis: When calling factorial with a smaller n, it would return the factorial of that smaller n.

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)
```



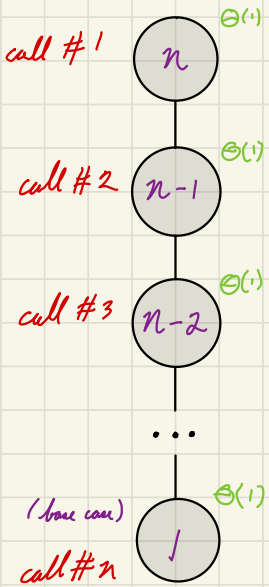

```python
# n >= 1
def factorial(n):
    if n == 1:
        return 1
    else:
        result = n * factorial(n - 1)
        return result
```

Recursive Trees

1. Represent each recursive call as a node ("leaf"). Inside of each node, write the size of the current input (e.g. n, $n-1$, $n/2$, etc).
2. If function call A makes function call B, we draw an edge ("branch") from A to B.
3. Next to each call, write the cost of the function as if the recursive call didn't exist. (local cost)

```python
# n >= 1
def factorial(n):
    if n == 1:          Θ(1)
        return 1        Θ(1)
    else:
        result = n * factorial(n - 1)
        return result   Θ(1)
```

$\Theta(1) + \Theta(1) = \Theta(1)$



$\therefore T(n) = \Theta(n)$

Counting Occurrences V.1

- Base case: if len(lst) == 0, return 0
- Hypothesis: When calling count_occ w/ smaller list, it will count occurrences in that smaller list.

```
def count_occ(lst, val):
    if len(lst) == 0:          Θ(1)
        return 0               Θ(1)
    else:
        first = lst[0]         Θ(1)
        rest = lst[1:]         Θ(i)

        if first == val:       Θ(1)
            return 1 + count_occ(rest, val)
        else:
            return count_occ(rest, val)
```

$\Theta_{local}(n)$

$\therefore T(n) = \Theta(n^2)$

```
def count_occ_v2(lst, val):
    def helper(lst, low, high, val):
        if low == high:
            if lst[low] == val:
                return 1
            else:
                return 0
        else:
            if lst[low] == val:
                return 1 + helper(lst, low+1, high, val)
            else:
                return helper(lst, low+1, high, val)
    if len(lst) == 0:
        return 0
    else:
        return helper(lst, 0, len(lst-1), val)
```

Power

- Iterative:

$$n^a = \prod_{i=1}^{a} n$$

- Recursive:

$$a^n = \underbrace{a * a * a * \cdots * a}_{a^{n-1}} * a$$

$$= a^{n-1} * a$$

$$= \begin{cases} a & n = 1 \\ a^{n-1} * a & n > 1 \end{cases}$$

- Base Case: If n is 1 return a.
- Recursive Hypothesis: When calling power w/ smaller n, it would return a raised to that smaller n.

```
def power(a, n):
    if n == 1:                  Θ(1)
        return a  # base case   Θ(1)
    else:
        rest = power(a, n-1)
        return rest * a         Θ(1)
```

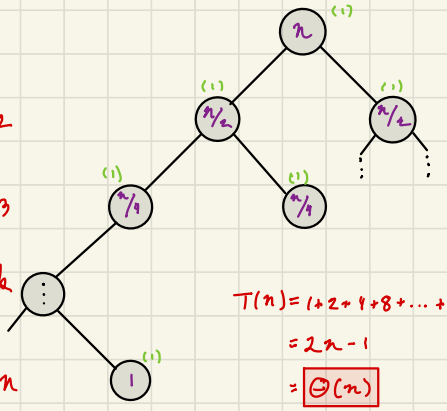
$\therefore T(n) = \Theta(n)$

$$a^n = a^{n/2} * a^{n/2} \quad \text{for even } n$$

$$= a * a^{(n-1)/2} * a^{(n-1)/2} \quad \text{for odd } n$$

```
def power(a, n):
    if n == 1:                          # 1
        return a                        # 1
    else:
        part_1 = power(a, n // 2)
        part_2 = power(a, n // 2)
        if n % 2 == 0:                  # 1
            return part_1 * part_2      # 1
        else:
            return a * part_1 * part_2  # 1
```

#1
#2
#3
#k
#n



$T(n) = 1 + 2 + 4 + 8 + \dots + n$

$= 2n - 1$

$= \Theta(n)$

$1 + 2 + 4 + \dots + n$

```
def power_opt(a, n):
    if n == 1:
        return a
    else:
        part = power_opt(a, n // 2)
        if n % 2 == 0:
            return part * part
        else:
            return a * part * part
```